



VIT[®]
BHOPAL

Programming

In Java

Project Report: *Campus Course &RecordsManager (CCRM)*

Session:2025-2029

Name: Shivangi Barthwal

Registration no:25BAI11395

Branch: B.tech CSE (AIML)

Course Code : CSE2006

INTRODUCTION

Campus Course & Records Manager (CCRM) is a Java-based console application designed to make academic record management easier and more organized. It allows users to manage student details, courses, enrollments, grades, and GPA records from one place.

The system provides simple options to add, search, update, and manage records, along with features like CSV import/export, backup, restore, and report generation. The project also applies important Java concepts such as OOP, collections, interfaces, exception handling, streams, and file handling to solve a practical academic problem.

Problem Statement

Managing student, course, enrollment, and grade records separately can be time-consuming and may lead to missing or inconsistent information. Tasks like checking enrolled credits or calculating academic performance often require going through multiple records.

CCRM brings these records together in one console-based system. It also uses simple validation and rules, such as checking credit limits before enrollment. This helps reduce manual effort and makes academic record management more organized and reliable.

Functional Requirements

- Student Management: Add, update, search, view, activate, or deactivate student records.
- Course Management: Create and manage course details, including their active status.
- Enrollment: Enroll students while checking duplicate entries and credit limits.
- Grade Management: Add or update grades for enrolled courses.
- GPA Calculation: Calculate and display a student's GPA based on recorded grades.
- Search & Update: Quickly find records and make necessary changes.
- CSV Handling: Import existing data and export records as CSV files.
- Backup & Restore: Save backup copies and restore data when needed.

- Reports: Generate simple academic summaries and statistics.
- Validation: Check inputs and prevent incomplete or invalid data.

Non-Functional Requirements

These requirements describe the overall quality and usability of the CCRM system.

- Usability: Menus and instructions should be simple and easy to follow.
- Reliability: The system should handle invalid ~~inputs~~ **inputs**, duplicate records, and file properly.
- Maintainability: The code should be well-structured and easy to update or modify.
- Performance: Searches, calculations, and file ~~operations~~ **operations** should work quickly for expected data size.

- Portability: The application should run on any system with a compatible JDK.

helpData Integrity: Validation should prevent incorrect, incomplete, or duplicate records.

- Recoverability: Backup and restore options should help recover important academic data when needed.

System Architecture

CCRM uses a simple layered structure suitable for a Java console application. The user interacts with the Console Menu, which sends requests to the Business Logic. This layer manages the required operations and works with the Model Classes and file-handling components.

Basic flow:

User → Console Menu → Business Logic →
Model Classes → File/CSV Storage

The model classes store entities such as Student, Course, Enrollment, and Grade, while service classes handle operations and rules.

File components manage tasks like reading, writing, CSV import/export, and backups. This separation keeps the application organized and easier to maintain.

Design Diagrams

The main user of CCRM can manage students and courses, enroll students, record grades, calculate GPA, handle files, and generate reports.

Diagram:

User → Students / Courses / Enrollment /
Grades / GPA / Files / Reports

Workflow Diagram

The basic flow of the application is:

Start → Menu → Select Option → Validate Input
→ Perform Operation → Show Result → Menu →
Exit

Sequence Diagram

The user selects an option from the menu. The request is passed to the required service, which processes the data and sends the result back to the menu for display.

Flow:

User → Menu → Service → Data/File Manager →
Service → Menu → User

Class/Component Diagram

The system contains service classes for students, courses, enrollment, and grades. Model classes such as "Student", "Course", "Enrollment", and "Grade" store the main

academic information. Other components handle files, backups, and reports.

ER Diagram

Students and courses are connected through enrollment records. Grades are linked to the corresponding enrollment.

Relationship:

STUDENT 1 — M ENROLLMENT M — 1 COURSE
ENROLLMENT 1 — 0/1 GRADE

DIAGRAM PLACEHOLDER

Insert the Use Case, Workflow, Sequence, Class, and ER diagrams here

Design Decisions & Rationale

The project uses a console interface to keep the focus on Java programming and application logic without the extra complexity of a graphical interface.

Separate classes are created for students, courses, enrollments, and grades so that each class has a clear purpose.

Collections are used to store records, while streams and lambda expressions make searching and filtering easier.

CSV and local files were chosen for storage because they are simple and suitable for a project of this size. A database can be added later.

Validation and exception handling help prevent incorrect data and keep the application running when users enter invalid information.

Implementation Details

CCRM is developed using Java classes, interfaces, enums, collections, and utility classes. The "Main" class starts the application and connects the user with the menu.

The main academic entities are represented using "Student", "Course", "Enrollment", and "Grade" classes. Encapsulation is used to control access to their data.

Collections store records during execution, while the Stream API and lambda expressions help with searching, sorting, and filtering.

Java File I/O and NIO.2 are used for reading and writing files. CSV files are used for importing and exporting records, while backup and restore provide basic data recovery.

Exception handling is included to manage problems such as invalid input, duplicate records, missing files, and incorrect enrollment requests.

Screenshots / Results

```
package ccrm;
```

```
=== Student Management ===
1. Add Student
2. List All Students
3. Update Student
4. Deactivate Student
5. View Student Transcript
6. Search Students
7. Back to Main Menu
Enter your choice: 3

--- Update Student ---
Student ID to update: 101
New Full Name (press Enter to skip):
New Email (press Enter to skip):
Student updated successfully!

=== Student Management ===
1. Add Student
2. List All Students
3. Update Student
4. Deactivate Student
5. View Student Transcript
6. Search Students
7. Back to Main Menu
Enter your choice: 1

--- Add New Student ---
Student ID: 51003
Registration Number: 25BAI10870
Full Name: Palak Gupta
Email: palak@vithhopal.ac.in
Student added successfully!

=== Student Management ===
1. Add Student
2. List All Students
```

```
package ccrm;
```

```
=== Student Management ===
1. Add Student
2. List All Students
3. Update Student
4. Deactivate Student
5. View Student Transcript
6. Search Students
7. Back to Main Menu
Enter your choice: 2

--- All Students ---
Student[ID: 5001, Reg: 2025001, Name: John Doe, Email: john.doe@university.edu, Active: Yes]
Student[ID: 5002, Reg: 2025002, Name: Jane Smith, Email: jane.smith@university.edu, Active: Yes]
Student[ID: 5003, Reg: 2025003, Name: Bob Johnson, Email: bob.johnson@university.edu, Active: Yes]
Student[ID: 101, Reg: 25BAI10870, Name: Palak Gupta, Email: palak.gupta@university.edu, Active: Yes]
Student[ID: 51003, Reg: 25BAI10800, Name: Ambar Gupta, Email: ambar.gupta@university.edu, Active: Yes]
Student[ID: 5003, Reg: 25BCE10404, Name: Ayushman Singh Bhaduria, Email: ayushman@gmail.com, Active: Yes]

=== Student Management ===
1. Add Student
2. List All Students
3. Update Student
4. Deactivate Student
5. View Student Transcript
6. Search Students
7. Back to Main Menu
Enter your choice: 3

--- Update Student ---
Student ID to update: 5001
New Full Name (press Enter to skip): John
New Email (press Enter to skip):
Student updated successfully!
```

```
=== Student Management ===
1. Add Student
2. List All Students
3. Update Student
4. Deactivate Student
5. View Student Transcript
6. Search Students
7. Back to Main Menu
Enter your choice: 5
```

```
--- Student Transcript ---
Student ID: S001
Transcript for: John
Registration No: 2023001
GPA: 0.00
```

Courses:

```
=== Student Management ===
1. Add Student
2. List All Students
3. Update Student
4. Deactivate Student
5. View Student Transcript
6. Search Students
7. Back to Main Menu
Enter your choice:
```

Testing Approach

Testing was performed by checking the main features with both valid and invalid inputs.

- Student: Add, search, update, and validate student records.
- Course: Add, search, update, activate, and deactivate courses.
- Enrollment: Test valid enrollment, duplicate enrollment, and credit-limit cases.
- Grade/GPA: Enter grades, update them, and check the calculated GPA.
- Files: Test CSV import/export and backup/restore operations.
- Invalid Input: Enter wrong formats, empty fields, and invalid menu choices.

Test Case| Action| Expected Result

Student| Add valid student| Record saved

Duplicate| Enter same ID| Duplicate rejected

Enrollment| Enroll in valid course| Enrollment successful

Credit Limit| Exceed credit limit| Enrollment rejected

Grade/GPA| Enter grades| GPA calculated

CSV| Export records| CSV file created

Backup| Restore data| Data recovered

Invalid Input| Enter wrong format| Error shown, app continues

Challenges Faced

One major challenge was keeping the relationship between students, courses, enrollments, and grades correct.

Credit-limit checking also required careful calculation of the student's existing enrolled credits before accepting a new course.

File handling was another area that needed attention because imported files may contain missing or incorrectly formatted information.

Combining different Java concepts into one project was also challenging. The solution was

to use each concept only where it was actually useful .

Creating a simple menu flow was important so that users could move between different operations without getting confused.

Learnings & Key Takeaways

This project provided practical experience in developing a complete Java application using multiple classes.

It helped improve the understanding of OOP concepts, including encapsulation, inheritance, abstraction, and polymorphism.

Working with collections, streams, and lambda expressions gave practical experience in handling and processing records.

The project also improved understanding of exception handling, validation, file operations,

CSV processing, backup, and data recovery.

Most importantly, it showed how dividing a program into separate classes and responsibilities can make the code easier to understand and maintain.

Future Enhancements

CCRM can be improved further by adding:

- Database support such as MySQL for handling larger amounts of data.
- GUI or web interface for easier interaction.
- Login and role-based access for students, faculty, and administrators.
- Semester-wise transcripts and detailed academic reports.
- Attendance and prerequisite tracking.
- Notifications for grades and enrollment updates.
- Dashboard and charts for academic statistics.

- Automatic backups and improved recovery options.
- Advanced search and filtering for larger datasets.

REFERENCES

- VITYARTHI
- My Programming JavaFaculty
Dr. Sandeep Monga
- Oracle Java Documentation – Java language, APIs, collections, exceptions, and file handling.
- Oracle Java Documentation – OOP, interfaces, inheritance, streams, and lambda expressions.
- Java Platform Documentation – JDK, JVM, and Java development features.